Drift Detection, Diagnosis and Reliability Analysis for Deployed Machine Learning Models

Sanjay Mirchandani[1*], Nathan Cherian[1], Rahul Guhagarkar[1], Alok Pal[1], Akul Patre[1]

[1] Department of Computer Engineering, Vivekanand Education Society's Institute of Technology, Hashu Advani Memorial Complex, Collector's Colony, Chembur, Mumbai 400074, India

[*] Corresponding author, e-mail: sanjay.mirchandani@ves.ac.in

Abstract

A drift alarm reports that production data have changed. The appropriate response, however, depends on the type of change, which current monitoring tools rarely identify: covariate shift calls for reweighting, prior shift for recalibration and concept drift for new labels, while label noise points to the labeling process rather than to the model. We present an open monitoring pipeline that detects drift, estimates its cost in accuracy without labels, locates the shifted features, names the drift type, scores the reliability of individual predictions and advises whether a cheap retrain will help. Its core is a procedure that names the four drift types with a stated error rate. Established label-free tests separate covariate shift from prior shift, while a new test on a few hundred late labels separates concept drift from label noise by asking whether the errors have changed beyond what the model's confidence explains. On injected drift in two census tasks and two model families, the procedure identifies the type in 85.7 to 94.3% of windows and outperforms rules used in monitoring practice. A learned classifier does better on the drift it was trained on but reaches only 31.1 to 51.2% on held-out drift mechanisms, where our tests retain 62.4 to 81.5%. On four real datasets with documented changes, the pipeline reported no drift on any control window and traced the known events.

Keywords

concept drift, label shift, model monitoring, drift diagnosis, hypothesis testing

1 Introduction

A deployed classifier can lose accuracy quietly as production data move away from the data it was trained on, often long before reliable labels reveal the loss [1]. Existing monitors are good at announcing that a distribution has changed [1] and at estimating how much accuracy has been lost [2, 3]. They rarely report what kind of change occurred, although each kind of drift calls for a different response.

The literature distinguishes three types of dataset shift [4]. Under covariate shift the inputs move while the rule linking inputs to labels stays the same, so reweighting the training data can help. Under prior shift only the class balance changes and recalibrating the model is usually enough. Concept drift alters the rule itself and needs fresh labels. We add label noise as a fourth type. It corrupts the labels used to judge the model and points to the labeling process rather than to the model.

Naming the type turns out to be harder than detecting a change. Covariate and prior shift both move the inputs and the predictions, whereas concept drift and label noise leave the inputs untouched and are therefore invisible to any label-free test.

We identify the type with a short sequence of hypothesis tests run at one significance level, with the family-wise error rate controlled. The first stage needs no labels and builds on established methods. It tests the inputs and the predictions for a shift, estimates the new class balance with black-box shift estimation (BBSE) [5] and, following Roschewitz et al. [6], asks whether this balance alone explains the shift in the inputs (prior shift) or not (covariate shift). The second stage is new. Using a few hundred late labels, it asks whether accuracy has fallen and, if so, whether the errors have changed in a way that depends on some input feature. Uniform label noise changes the error rate only according to how confident the model is, whereas concept drift changes it within one region of the input space. Since every step is a formal test, the false alarm rate is fixed in advance.

The contributions are as follows.

1 An audit test that separates concept drift from label noise using only a few hundred late labels. Existing shift tests, such as DetectShift [7], either require labeled production data or ignore label noise.

2 A hierarchical procedure that combines this audit with a label-free stage and names four drift types under family-wise error control.

3 A benchmark covering two tabular tasks and two model families, with matched as well as held-out drift mechanisms. It shows that tests built on the

definitions of the types transfer to new drift, whereas a learned rule does not.

4 An open end-to-end monitoring pipeline that covers detection, accuracy estimation, root cause analysis, diagnosis, reliability scoring and remediation triage, evaluated on four real drifts with documented causes.

Section 2 reviews related work and Section 3 describes the pipeline. Section 4 sets out the experiments and Section 5 reports them. Section 6 discusses what the results mean and where they stop. Section 7 concludes.

2 State of the art

Table 1 compares our work with the closest existing methods: black-box shift detection (BBSD), BBSE, confidence-based performance estimation (CBPE), DetectShift and the method of Roschewitz et al. The paragraphs after it review what each group of related work offers a monitoring team and what it leaves open.

Table 1 Capabilities of the closest methods and of ours. A tick marks a capability that the method provides

Method	Error rate stated	Accuracy estimate	Localizes features	Types named	Concept vs noise	Held-out drift
BBSD [1]	√	–	limited	–	–	–
BBSE [5]	√	–	–	prior	–	–
CBPE [3]	–	√	–	–	–	–
DetectShift [7]	√	–	–	needs labels	–	–
Roschewitz et al. [6]	√	–	–	prior, cov.	–	–
Ours	√	√	√	four	√	√

Drift types. Moreno-Torres et al. [4] define covariate shift, prior shift and concept shift according to which part of the joint distribution of inputs and labels changes. DetectShift tests for a shift in this joint distribution and in each of its parts, $P(X)$, $P(Y)$, $P(X \mid Y)$ and $P(Y \mid X)$ [7]. However, it requires labels for the new data, a limitation its authors acknowledge. We adopt the same definitions, add label noise as a fourth type and require labels only for a small audit.

Drift detection. Detectors that track the error rate, such as the drift detection method (DDM) [8], need a label for every prediction. They can detect concept drift but cannot distinguish it from label noise. Label-free detectors instead compare the inputs or the model outputs. Rabanser et al. [1] compared univariate tests, kernel tests [9], classifier two-sample tests [10] and BBSD applied to the model outputs. They found that BBSD worked best overall. All of them indicate that a shift has occurred but not of which type. Our pipeline uses such tests as building blocks and goes on to name the type.

Label shift. BBSE recovers the new class prior from the confusion matrix of any classifier [5]. It assumes that the inputs within each class do not change, which is precisely the assumption that separates prior shift from covariate shift. Roschewitz et al. [6] test this assumption on medical images. They resample the reference data to the estimated prevalence and attribute the shift to prevalence if the features then no longer differ. Our label-free stage builds on the same idea but uses importance weights instead of resampling and places every label-free decision in one family with a stated error rate.

Performance estimation without labels. Average confidence (AC) [11], difference of confidences (DoC) [12], average thresholded confidence (ATC) [2] and CBPE [3] all estimate accuracy from model outputs. Each assumes that the relationship between confidence and accuracy carries over to production. Because models often remain confident as their accuracy falls under shift [13], this assumption can fail. Our pipeline reports CBPE alongside the other three and treats it as an upper bound when the assumption breaks.

Explanation, reliability and benchmarks. Shapley additive explanations (SHAP) [14] and class activation maps (CAM) [15] attribute predictions to features or to image regions. Out-of-distribution (OOD) scores such as the Isolation Forest [16] flag unusual inputs, while importance weighting corrects covariate shift [17–19]. Folktables [20] and CLEAR [21] provide real shifts, although the type of each shift is unknown. Injected shifts supply a ground truth [1] but risk favoring the method under test. Each of these tools serves one stage of our pipeline. Our benchmark holds out drift mechanisms to limit that risk.

Each prior method thus addresses part of the problem. To our knowledge, no published method separates concept drift from label noise with a small label audit. Nor has any type diagnosis been evaluated on drift mechanisms held out from its design.

3 Method

The pipeline is designed for a team that has a deployed model, the labeled data on which it was built and a stream of unlabeled production data. Nothing else is required, although a few hundred late labels sharpen the diagnosis once they arrive. We first define the four drift types and

then follow the data through the pipeline, whose stages Fig. 1 shows.

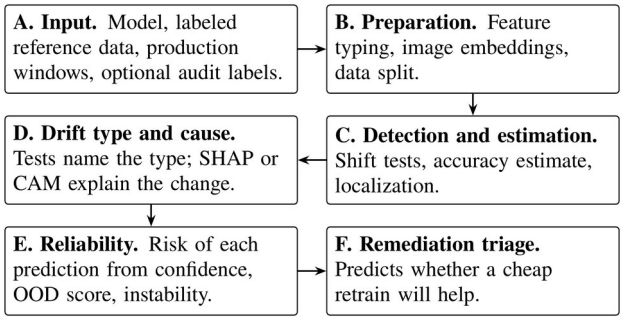


Fig. 1 Stages of the monitoring pipeline, from the inputs (A) to the advice on retraining (F)

Stages A to C bring the data into a common form, test whether anything has changed and estimate how much accuracy the change may cost. Stage D, the core of this paper, names the type of change and points to its likely cause. Stages E and F turn these findings into action for individual predictions and for the retraining decision. Only the optional audit in stage D uses production labels. Apart from the population stability index (PSI), which alarms at its conventional threshold, every decision is a hypothesis test at the significance level $\alpha = 0.05$. The family-wise error rate is controlled by the Holm procedure [22], its weighted form or its Bonferroni bound.

3.1 Notation and drift types

The tests described below require a precise statement of what each type of drift changes. The reference data are drawn from a distribution $P(X, Y)$ over inputs X and labels Y. A production window is a batch of rows from a later period or a different region, drawn from $Q(X, Y)$. The model returns a predicted label $\hat{y}$ and a confidence c, its largest class probability. The four types differ in what changes between P and Q. Covariate shift has $Q(X) \neq P(X)$ while $Q(Y \mid X) = P(Y \mid X)$. Prior shift has $Q(Y) \neq P(Y)$ while $Q(X \mid Y) = P(X \mid Y)$. Concept drift has $Q(Y \mid X) \neq P(Y \mid X)$. Under label noise the true distribution stays the same, but each observed label differs from the true one with some probability. An audit is a random sample of m production rows whose labels arrive late. A control window is drawn from the reference distribution itself and should therefore raise no alarm. Real drift can mix several types, as Section 6 discusses.

3.2 Reference split, detection and estimation

Stage C addresses three questions in turn: whether the data have moved, how much accuracy the change might cost and where it is located. It first requires a clean reference. The labeled reference data are split into three disjoint parts, stratified by class: 60% for training, 20% for calibration and 20% for testing. The model is fitted on the training part alone. The calibration part is the reference window for every detector and fits the accuracy estimator, while the test part gives the reference accuracy and acts as the control window. Nominal features, such as an occupation code, are treated as categorical because their numeric order carries no meaning. Images are mapped to the 512-dimensional output of the global average pooling layer of a frozen residual network with 18 layers (ResNet-18) trained on ImageNet [23], so image drift can be tested with the tabular tests [1].

Six detectors decide whether the inputs or the model outputs have moved. The Kolmogorov–Smirnov (KS) test is applied to each numeric feature with a Bonferroni correction. PSI alarms at the conventional value of 0.2 [24]. The maximum mean discrepancy (MMD) test [9] is a permutation test on all standardized inputs jointly, using a Gaussian kernel. The confidence test compares histograms of c with a χ^2 test of homogeneity, pooling sparse cells until every expected count reaches 5 [25]. Two detectors from Rabanser et al. [1] serve as baselines. BBSD applies a KS test to each class probability. The domain classifier trains gradient boosted trees to separate 3,000 reference rows from as many production rows and compares its held-out accuracy with $1/2$ by an exact binomial test [10]. Every permutation p value is given by $(1+b)/(1+B)$, where b of the B permutations reach the observed statistic [26].

Because labels arrive late, accuracy is first estimated from the model's own confidence with CBPE [3]. A map $a(\cdot)$ from confidence to the probability of a correct prediction is fitted by histogram binning on the calibration part. The estimate $\hat{A}$ for a window with confidences c_1 to c_n is given by

$$\hat{A} = \frac{1}{n} \sum_{i=1}^{n} a(c_i). \tag{1}$$

A confidence bin with fewer than 5 calibration rows gives no reliable accuracy, so predictions in such a bin keep their own confidence in Eq. (1). AC, DoC and ATC (with the negative entropy score) are fitted on the same calibration part as baselines [2, 11, 12].

An alarm is more useful when it also says where the change is located. Localization tests every feature with KS (numeric) or χ^2 homogeneity (nominal) under Holm

correction and reports an effect size in $[0,1]$, the KS statistic or Cramér's V. For nominal features it also reports the share of production rows whose category never occurs in the reference.

3.3 Drift type diagnosis

Stage D turns an alarm into one of the four types, each with its own response. It works in two stages along the decision path in Fig. 2, which the rest of this section explains. The upper row of questions uses no labels and separates covariate from prior shift. The lower row, which runs only when no label-free test rejects and an audit is available, separates concept drift from label noise.

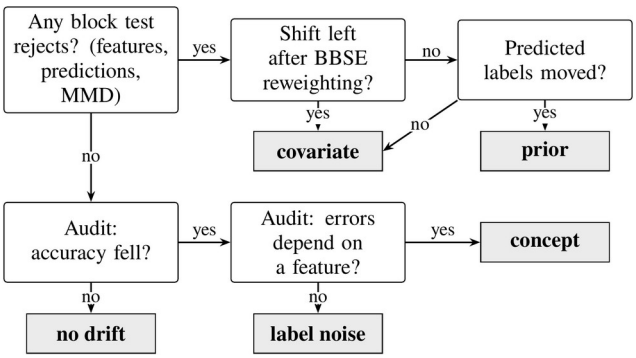


Fig. 2 Decision path that names the drift type. Questions are shown in white boxes and verdicts in gray boxes; the upper row needs no labels and the lower row uses the audit

Label-free stage. The first question in Fig. 2 is whether anything has moved at all. Three blocks of tests answer it: the feature tests, a χ^2 test on the predicted labels and the MMD test. They form one family, tested at level α with a weighted Holm procedure [22]. The three blocks receive equal weight, which the feature block divides equally among its features. Whenever a test is rejected, its weight passes to the tests that remain, so the family-wise error rate is controlled over every label-free decision. The first step equals the Holm procedure over the three blocks, with each block's features pooled by their Bonferroni p value. Pooling lets the MMD test, whose smallest possible p value is $1/(B+1)$, compete with many feature tests. If no test rejects, the label-free stage reports no shift.

If a block rejects, the diagnosis asks whether a change in the class balance alone explains the shift. BBSE estimates the new class prior. Let $C_{jk}=P(\hat{y}=j, y=k)$ on the calibration part and let μ_j be the share of production predictions in class j. The class weights $w_k=Q(Y=k)/P(Y=k)$ are given by the solution of

$$C\,w=\mu. \tag{2}$$

Under a pure prior shift the inputs within each class keep their distribution, so reweighting each reference row by the weight w_y from Eq. (2) should reproduce the production inputs. This label shift consistency test follows the prevalence adjustment of Roschewitz et al. [6]. The feature tests are repeated against the reweighted reference with Kish's effective sample size [27] and Holm correction. Any feature that still differs carries a covariate shift. If none differs and the predicted labels have moved, the shift is a prior shift. A prior shift always moves the predicted labels of an informative classifier, so a shift without such a move is called covariate.

Audit stage. Concept drift and label noise leave the inputs and the model outputs unchanged, so only labels can reveal them. The audit draws m late labels and first checks with a one-sided two-proportion test whether accuracy has fallen. If it has, the diagnosis asks whether the change in errors depends on a feature. A logistic model of whether a prediction is correct is fitted on the calibration rows and the audit rows together. Its null form contains 5 confidence bins at reference quantiles, bins of a feature f, an audit indicator and the interaction of that indicator with the confidence bins. The alternative adds the interaction of the audit indicator with the bins of f and is compared with the null by a likelihood ratio test for each feature. A nominal category receives its own bin only if it occurs at least n_{min} times in the audit, where n_{min} is given by $\lceil 5/min(e,1-e) \rceil$ and e is the audit error rate. The remaining categories share one bin.

It is the confidence term that separates the two types. In a binary task, uniform label noise at rate r makes the probability of a correct prediction equal to $r+(1-2r)q(x)$, where $q(x)$ is the model's own probability of being right at x. The change therefore depends on x only through $q(x)$, which the confidence bins of the null absorb. Concept drift instead shifts the errors within a region of some feature. A feature that remains significant after Holm correction therefore indicates concept drift, whereas no significant feature indicates label noise. Because each of the two stages is tested at level α, a window without drift receives a drift verdict with probability at most 2α.

3.4 Root cause analysis

Knowing the type tells a team what to do, while root cause analysis tells it where to look. This step asks which inputs the model's decisions depend on differently after the shift. It narrows the search but does not prove a cause. For

tabular models we compute SHAP values [14] of the deployed model with its preprocessing and compare the mean absolute attribution of each feature between reference and production rows. Each change carries a percentile bootstrap interval whose level is Bonferroni adjusted over the features. For images, CAM shows which image regions support the prediction of the linear head [15].

3.5 Reliability and remediation triage

The tests so far describe a window as a whole. Two practical questions remain: which individual predictions deserve less trust and whether a cheap retrain would recover the lost accuracy. For the first, each prediction receives three signals in $[0, 1]$: the confidence risk $1 - c$, an OOD score and its instability under small input perturbations. The OOD score combines an Isolation Forest [16] with a standardized distance. A logistic regression fitted on the calibration part combines the three signals into a probability of error. It is fitted only when the calibration part holds at least ten wrong predictions per signal [28]; otherwise the risk is simply $1 - c$.

For the second, the triage predicts without production labels whether a cheap retrain will help. A classifier that separates reference from production rows provides density ratio weights, fitted on separate folds and truncated at 20 times their mean [19]. A model trained with these weights is compared with an unweighted one on a held-out 30% of the reference, itself weighted by the density ratio [17, 18]. When late labels from an earlier window are available, a second probe retrains on them and is validated on a held-out part of that window. A retrain is recommended when a probe's predicted gain is significant after Holm correction across probes.

4 Experimental setup

The experiments answer two questions. Does the procedure name the type correctly when the truth is known, including for drift it was not designed for? And does it behave sensibly on real drift, where the type is unknown but the causes are documented?

Protocol. All settings below were fixed before the evaluation that uses them. The diagnosis was frozen before the held-out design B was first run. A later code review uncovered two errors in the diagnosis. The label-free stage had used a gatekeeping rule whose family-wise error rate reached 6.6% at a nominal 5% in a simulation, compared with 4.9% for the weighted Holm procedure of Section 3.3. The audit had also counted nominal categories over the reference and audit rows together. Both were corrected without regard to the results and every result was regenerated, which changed the verdict in 115 of the 5,300 benchmark windows. Permutation tests use 200 permutations, SHAP explains 1,000 rows per side against 100 background rows and each real audit uses 600 labels from its own random stream. All randomness is seeded, so every result repeats exactly. One workstation with a 20-core processor and a graphics processor ran everything. The software was scikit-learn [29], SciPy, NumPy, SHAP, OpenCLIP and PyTorch.

4.1 Models

Since the procedure should not depend on one kind of model, the same pipeline monitors two untuned model families. The first is logistic regression (LR) with L_2 regularization ($C = 1$) on standardized numeric and one-hot encoded nominal features. The second is histogram gradient boosting (GB) [30] from scikit-learn [29] with default settings, no early stopping and native splits on nominal codes. Codes beyond the 254 most frequent are pooled, which is the limit of the implementation. For CLEAR-10 both models operate on the frozen embeddings. SHAP is reported for LR only, because for GB the exact explainer would take hours per window.

4.2 Datasets

Each of the four datasets contains drift with a documented cause, so the verdicts can be checked against known events. Table 2 summarizes them. The two census tasks come from the American Community Survey (ACS) and use data from California (CA), Texas (TX), New York (NY), Florida (FL) and Washington (WA).

Table 2 Datasets, drift windows and reference accuracy. Rows gives the smallest and largest window, Inputs the number of features or embedding dimensions and the last two columns the reference test accuracy of LR and GB

						Ref. accuracy	
Dataset	Reference	Windows	Rows	Inputs	Classes	LR	GB
ACS Income	CA 2014	CA 2015–19 and 2021–23; TX, NY, FL, WA 2014	36,162–203,278	10	2	0.826	0.833
ACS Employment	CA 2014	CA 2015–19 and 2021–23; TX, NY, FL, WA 2014	56,576–326,241	16	2	0.729	0.772
Gas sensor	batches 1–2	batches 3–10	161–3,613	128	6	0.982	0.985
CLEAR-10	bucket 1	buckets 2–10	3,297–3,300	512	11	0.914	0.897

ACS Income and ACS Employment. Both tasks come from the ACS of the United States Census Bureau, as packaged by Folktables [20]. ACS Income predicts whether a person earns more than $50,000 from 10 features. ACS Employment predicts, from 16 features, whether a person aged 16 to 90 is employed. With CA 2014 as the reference, the temporal windows are CA 2015 to 2019 and 2021 to 2023, as there is no standard 2020 release [31]. The geographic windows are TX, NY, FL and WA in 2014. In 2019 the relationship variable RELP was replaced by RELSHIPP [32], which we map back to RELP using the code labels of the 2018 and 2019 data dictionaries (Table S2 in the Supplement). Five documented changes fall within these years: new occupation codes in 2018 [33], a redesigned relationship question in 2019, a new coding of race answers from 2020 [34], the pandemic and wage growth against the fixed income threshold.

Gas sensor array drift. This dataset from the University of California, Irvine (UCI) machine learning repository holds 13,910 measurements of six gases by 16 metal oxide sensors over 36 months, in ten batches [35, 36]. Each measurement has eight features per sensor. As the sensors age, their response to the same gas drifts while the labels stay exact, a documented change in $P(X \mid Y)$. Batches 1 and 2 form the reference and batches 3 to 10 the windows. Because some batches are small, the number of audit labels for a window of n rows is given by $min(600, \lfloor n/2 \rfloor)$.

CLEAR-10. This image stream holds ten object classes and a background class in ten time-ordered buckets of about 3,300 images each [21]. Bucket 1 serves as the reference.

4.3 Injection benchmark

Real drift has no known type, so the diagnosis is graded on injected drift. The benchmark uses the 2014 data of 5 states (CA, TX, NY, FL, WA) for both ACS tasks. Each state is a reference distribution with its own model, split as above. For each of 10 seeds the split, the window and the injection are redrawn, which gives 50 replicates per configuration. Each replicate draws a production window of 4,000 rows and a grading holdout of 2,000 rows from the test part. Each mechanism is injected into both at three intensities. The audit uses 600 labels. Proportions carry Wilson intervals [37] and means carry bootstrap intervals over replicates.

Design A injects the four types in ways that partly match the tests. Covariate shift rescales two numeric features (ACS Income: AGEP and WKHP) for a random subset of rows. Prior shift drops rows of one class until the positive rate rises by 0.14, 0.23 and 0.32. Concept drift inverts the labels of the 15, 30 and 50% of rows with the highest value of one feature (ACS Income: SCHL). Label noise flips a random 8, 18 and 32% of labels. Each replicate also includes a control window without drift.

Design B uses mechanisms that the tests were not designed for. Covariate shift is selection bias: each row is kept with a probability given by $\sigma(\beta s(x))$, where σ is the logistic function, s is a seeded random score over all inputs and $\beta \in \{0.5, 1, 1.5\}$. The rows stay real and $P(Y \mid X)$ is exactly unchanged. Prior shift lowers the positive rate by 0.08, 0.14 and 0.20. Concept drift inverts the labels in the top 15, 30 and 50% of a second random score, which defines an oblique region that no single feature describes. For ACS Income a second concept drift raises the income threshold to $55,000, $60,000 and $70,000. Label noise is class-conditional, with positive labels flipped to negative at rates of 10, 20 and 35%.

4.4 Baselines and ablations

A diagnosis is only worth its cost if it beats simpler options. Every method names one of the five outcomes (no drift or one of the four types) from the same tests and audit labels. Three rules represent current practice. *Input monitoring* reads any input alarm as covariate shift, which is how drift alarms are commonly read. *BBSE first* reads a shift in the predicted labels as prior shift [1, 5] and any other alarm as covariate shift, which makes it our label-free stage without the consistency test. *Error-rate monitoring* also reads a significant audit accuracy drop as concept drift, like an error-rate detector [8].

The *learned meta-classifier* tests whether a learned rule could replace the tests. It is a GB model on nine statistics of each window: the evidence $-\log_{10} p$ of each block test, the share of features with residual shift, the BBSE prior change, the CBPE change, the evidence and size of the audit accuracy drop and the evidence of the error structure test. In design A it is validated by leaving one state out. In design B it is trained on design A without the predicted state, so it never sees a held-out mechanism. Three ablations remove the MMD block, remove the confidence term from the audit null or replace the confidence bins by cells of predicted class and confidence.

Our procedure is compared with each other method by the per-replicate difference in the share of correctly typed windows, tested by sign-flip permutation (10,000 flips) with Holm correction. Replicates are the unit of analysis because the windows of one replicate share a model and a reference.

5 Results

The benchmark results come first: how often the type is named, whether the procedure transfers to held-out drift and how well its building blocks work. Real drift follows.

5.1 Drift type against baselines

The central question is how often each method names the injected type. Table 3 gives the answer for every configuration, each with 50 replicates.

Table 3 Share of benchmark windows with the correct drift type (%), for each model family, task and design

	Inc. A	Inc. B	Emp. A	Emp. B
Logistic regression				
Ours	94.3	62.4	85.7	80.5
Input monitoring	30.3↓	20.0↓	30.5↓	25.0↓
BBSE first	30.5↓	27.7↓	30.5↓	39.2↓
Error-rate monitoring	52.3↓	48.5↓	52.2↓	62.8↓
Meta-classifier	97.5	31.1↓	95.8↑	48.2↓
Ours, no MMD	95.2	62.9	86.6	81.3
Ours, no confidence	79.1↓	62.9	83.7↓	80.7
Ours, class cells	93.1	65.6↑	86.2	91.8↑
Gradient boosting				
Ours	94.2	63.2	90.5	81.5
Input monitoring	30.3↓	20.0↓	30.5↓	25.0↓
BBSE first	30.5↓	28.1↓	30.6↓	40.3↓
Error-rate monitoring	52.5↓	49.6↓	53.1↓	64.8↓
Meta-classifier	98.5↑	39.2↓	97.4↑	51.2↓
Ours, no MMD	95.1	63.9	90.5	81.3
Ours, no confidence	78.0↓	62.9	85.4↓	82.0
Ours, class cells	92.9↓	66.5↑	90.6	93.7↑

Inc. and Emp. denote ACS Income and ACS Employment. A and B are the two designs. An arrow marks a method that differs significantly from ours after Holm correction: a down arrow means that ours is better and an up arrow that ours is worse. Within each model family the upper block contains the competing methods and the lower block the ablations of our procedure.

With LR on ACS Income, our procedure names the type in 94.3% of design A windows (95% interval [92.3, 95.8]), compared with at most 52.3% for a rule baseline. Each rule can name only some of the five outcomes, which caps its score. Across all eight configurations our procedure is significantly better in 28 of the 32 comparisons with a baseline or the learned rule and significantly worse in 3, all against the learned meta-classifier on design A (ACS Employment LR, ACS Income GB and ACS Employment GB).

The learned meta-classifier is the strongest competitor on design A. Validated on states it has not seen, it reaches 97.5% on ACS Income, above our 94.3%. On the held-out mechanisms of design B, however, the picture reverses. Trained on design A, the meta-classifier falls to 31.1% on ACS Income and 48.2% on ACS Employment, while our procedure retains 62.4% and 80.5%. The learned rule had tied each type to the size and shape of the injected shift. For example, it associated covariate shift with a large shift in two features and therefore misses selection bias, which moves many features slightly. This is the main argument for tests over a learned rule: the tests encode the definitions of the drift types rather than the appearance of the training examples.

The overall rates hide which types are confused with which, so Fig. 3 shows this breakdown for LR.

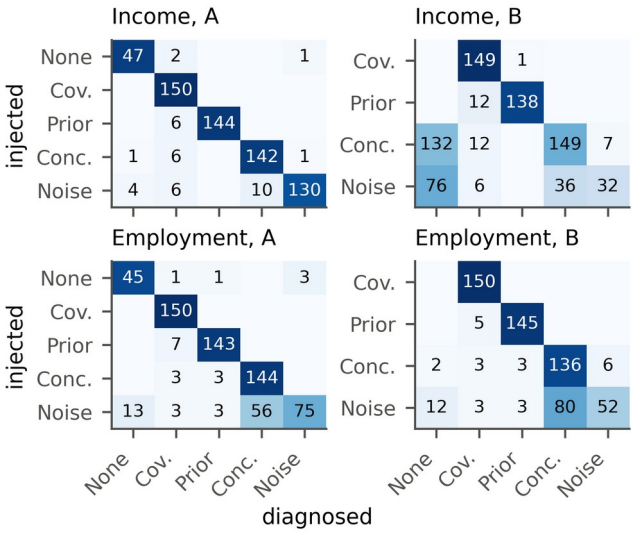


Fig. 3 Injected (rows) and diagnosed (columns) drift types on the benchmark with LR, in counts of windows; darker cells hold a larger share of their row

A perfect result would fill only the diagonal. Because design B has no control windows, false alarms are measured on design A. In design A most errors fall in the label noise row, where noise is read as concept drift or as no drift. The control row holds the false alarms. Our procedure raised them on 13 of 200 control windows (6.5%), 7 in the label-free stage and 6 in the audit, which is below the bound of 10% derived in Section 3.3. In design B the errors concentrate in the concept and noise rows, for reasons explained in Section 5.2.

The ablations show which parts of the procedure matter. Removing the confidence term from the audit null costs

15.2 points on ACS Income design A, because uniform label noise then looks like concept drift. Removing the MMD block makes little difference on the benchmark (−0.9 points), whose shifts are visible feature by feature, but its value becomes clear on the image embeddings in Section 5.4. Replacing the confidence bins by cells of predicted class and confidence helps on ACS Employment design B (91.8% against 80.5%) but costs accuracy on ACS Income design A (93.1%). Adopting it now would mean choosing the method after seeing design B, so it remains an ablation and a direction for future work.

5.2 Held-out mechanisms

Design B asks whether the procedure transfers to drift it was not designed for. It is the stronger of the two tests, since no part of the procedure was shaped by these mechanisms. Table 4 breaks the results down by mechanism.

Table 4 Design B results by drift mechanism

	ACS Income				ACS Employment			
	LR		GB		LR		GB	
Mechanism	Corr.	Drop	Corr.	Drop	Corr.	Drop	Corr.	Drop
Covariate: selection bias	99	0.2	99	0.3	100	−0.3	100	−0.3
Prior: decrease	92	−2.8	93	−2.4	97	2.5	97	1.9
Concept: oblique region	95	21.9	95	22.5	91	15.7	97	17.7
Concept: income threshold	4	−0.3	6	0.2	–	–	–	–
Noise: class-conditional	21	2.8	23	3.1	35	7.7	33	8.0

Corr. is the share of windows typed correctly (%) and Drop is the mean true accuracy drop in points, where a negative value means that accuracy rose. The income threshold mechanism exists only for ACS Income.

Three of the mechanisms transfer well. Selection bias is identified as covariate shift in 99% of ACS Income windows with LR, even though no value is distorted and the shift spreads over all features. A lower positive rate is identified as prior shift in 92%. Concept drift in an oblique region is found in 95%, although no single feature defines that region.

Two mechanisms do not transfer. Raising the income threshold relabels some rows in every window but barely changes accuracy (mean drop −0.3 points). Because the audit first asks whether accuracy fell, it reports no drift in 132 of the 150 windows. The procedure is designed to find drift that harms the model, which this drift does not. Class-conditional label noise is identified correctly in only 21% of ACS Income windows. At low rates the accuracy drop is too small to detect with 600 labels. At high rates the errors follow the true class, which correlates with the features, so the error structure test reads the pattern as concept drift. This is a genuine limitation of a null model that conditions on confidence alone. Both failures lie in the audit rather than in the label-free stage.

The same patterns hold for ACS Employment and for GB. On design A our procedure reaches 85.7% on ACS Employment with LR and, with GB, 94.2% on ACS Income and 90.5% on ACS Employment. Label noise is the hardest type on ACS Employment, identified in 50.0% of windows. The methods rank the same for both model families. The tests read only the model's outputs and its errors, so they need no modification for a nonlinear model.

5.3 Detection and accuracy estimation

The diagnosis builds on the detectors and the accuracy estimate, so both are also checked on the benchmark. Table 5 gives the alarm rate of each detector by injected type, pooled over the four design A configurations.

Table 5 Detector alarm rates on design A by injected type

	Alarm rate (%) by injected type				
Detector	No drift [95% CI]	Covariate	Prior	Concept	Noise
KS	1.0 [0.3, 3.6]	100.0	100.0	1.0	1.0
PSI	0.0 [0.0, 1.9]	99.7	0.3	0.0	0.0
MMD	7.0 [4.2, 11.4]	100.0	100.0	7.0	7.0
Confidence	5.0 [2.7, 9.0]	93.8	86.7	5.0	5.0
BBSD	2.5 [1.1, 5.7]	100.0	100.0	2.5	2.5
Domain classifier	6.0 [3.5, 10.2]	100.0	72.0	6.0	6.0

The no drift column is the false alarm rate with its 95% confidence interval (CI). Over 200 control windows every interval contains $\alpha = 0.05$ or lies below it. Every detector catches covariate shift and all except PSI also catch most prior shifts. Concept drift and label noise windows share their inputs with the control window of their replicate, so no detector can see them, which is why the audit is needed. Table 6 compares the four accuracy estimates by their mean absolute error against the measured accuracy.

Table 6 Mean absolute error of the label-free accuracy estimates (points)

	Benchmark (design A)				Real windows			
Estimator	None	Cov.	Prior	Conc., noise	Inc.	Emp.	Gas	CLEAR
CBPE	**0.50**	**9.20**	3.15	14.71	1.86	1.06	30.98	0.82
AC	0.83	9.58	3.65	**14.66**	1.87	1.25	30.36	5.32
DoC	**0.50**	9.63	3.12	14.71	1.77	1.06	29.78	0.76
ATC	0.63	10.31	**2.43**	**14.66**	**1.15**	**1.02**	**26.95**	**0.62**

Cov. and Conc. denote covariate shift and concept drift, Inc. and Emp. the two ACS tasks. The smallest error in each column is shown in bold. On design A the errors of all four estimates stay below 0.83 points without drift and below 10.31 under covariate shift. Under concept drift and label noise every error equals the whole loss, about 14.71 points, because the inputs do not change. Across these 8 comparisons, ATC has the smallest error in 6 and CBPE in only 2. On the gas sensor data all four are off by 26.95 to 30.98 points on average. In batch 8 the LR model's mean confidence falls from 0.973 to 0.595 and its accuracy to 0.330, yet CBPE estimates 0.618, so the estimate is best read as an upper bound on accuracy.

5.4 Real drift

Real drift has no known type, but its causes are documented. The question here is whether the verdicts agree with those causes and whether the control windows stay quiet. Fig. 4 shows how accuracy evolves over time in all four datasets.

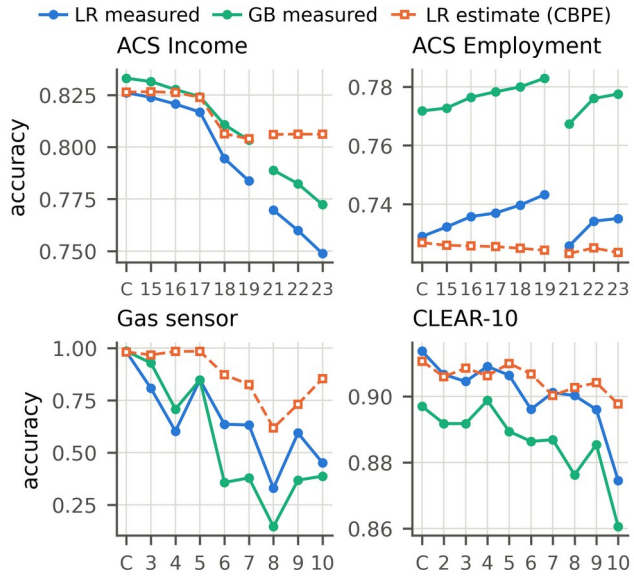


Fig. 4 Measured accuracy of both models and the label-free CBPE estimate on the temporal windows

C marks the control window and the numbers mark years (ACS), batches (gas sensor) or buckets (CLEAR-10). The gap in the ACS lines marks the missing 2020 release.

ACS Income shows the clearest trend, so we discuss it first. Its accuracy falls over time for both models, from 0.824 in 2015 to 0.749 in 2023 with LR, while the CBPE estimate falls much less. Table 7 gives the details behind the ACS Income panel of Fig. 4.

Table 7 Detection and diagnosis on the ACS Income windows. The alarms are those of the LR model: K is the KS test, M the MMD test, C the confidence test, B BBSD and D the domain classifier

	Accuracy				Alarms						
Window	LR	ΔLR	ΔGB	Est. err.	K	M	C	B	D	Top shifts	Type
Control	0.826	0.0	0.0	0.0	–	–	–	–	–	–	–
CA 2015	0.824	−0.2	−0.2	+0.3	√	–	–	–	–	OCCP	–
CA 2016	0.821	−0.5	−0.5	+0.6	√	–	–	√	√	OCCP, POBP	Cov.
CA 2017	0.817	−0.9	−0.9	+0.7	√	√	√	√	–	OCCP, POBP	Cov.
CA 2018	0.795	−3.2	−2.2	+1.2	√	√	√	√	√	OCCP, POBP	Cov.
CA 2019	0.784	−4.2	−3.0	+2.0	√	√	√	√	√	OCCP, RELP	Cov.
CA 2021	0.770	−5.6	−4.4	+3.6	√	√	√	√	√	OCCP, RAC1P	Cov.
CA 2022	0.760	−6.6	−5.1	+4.6	√	√	√	√	√	OCCP, RAC1P	Cov.
CA 2023	0.749	−7.7	−6.1	+5.7	√	√	√	√	√	OCCP, RAC1P	Cov.
TX 2014	0.802	−2.4	−2.7	+1.1	√	√	√	√	√	POBP, RAC1P	Cov.
NY 2014	0.804	−2.2	−2.7	+0.8	√	√	√	√	√	POBP, RAC1P	Cov.
FL 2014	0.786	−4.0	−4.3	+2.2	√	√	√	√	√	POBP, RAC1P	Cov.
WA 2014	0.801	−2.5	−2.5	+0.9	√	√	√	√	√	POBP, RAC1P	Cov.

Δ is the accuracy change in points and Est. err. the error of the CBPE estimate. A tick marks an alarm. Type is the verdict of the diagnosis, which is the same for LR and GB in every window (Cov. denotes covariate shift). Top shifts lists the two features with the largest effect size: occupation (OCCP), place of birth (POBP), relationship (RELP) and race (RAC1P).

The control window raises no alarm with either model. CA 2015, whose accuracy did not change significantly, is diagnosed as no drift by both models, while every other window is diagnosed as covariate shift. The documented events appear in the localization. When the occupation codes were revised in 2018, 26.6% of rows carried a code never seen in 2014. The model's reliance on occupation then fell (SHAP change −0.032). After the relationship question was redesigned in 2019, relationship shifts with an effect size of 0.091. From 2021 onward race shifts with an effect size of 0.186, following the change in how the Census Bureau codes written race answers [34]. Wage growth against the fixed threshold shows up in the labels. The share of positive labels rises from 0.355 in the control window to 0.498 in 2023, while the model predicts only 0.338. Part of the 2023 loss is therefore a change in the labeling rule, which no label-free test can see.

The other datasets differ in how much the drift costs. On ACS Employment every window is diagnosed as covariate shift, yet accuracy changes by at most 2.2 points with LR. The inputs do move, including relationship in 2019 and race from 2021, but the model is barely affected. A drift alarm alone would overstate the problem here, whereas the accuracy estimate and the audit put it in proportion. Every gas sensor batch is also diagnosed as covariate shift. With LR, accuracy changes by −65.2 points in batch 8. This matches the known cause: aging sensors change $P(X \mid Y)$. BBSE first labels every one of these batches as a prior shift because the predicted labels move as well, but the consistency test shows that they move because the sensors read differently, not because the mix of gases changed.

On CLEAR-10 the change is subtle. Both models report no drift in buckets 2 and 3 and covariate shift in buckets 4, 5, 6, 7, 8, 9 and 10. In buckets 4, 5, 6 and 9 no single embedding dimension shifts significantly with LR. Only the MMD test detects the change, while the predicted labels stay put. These buckets show why the multivariate block is needed. The only detector alarm on a control window comes from the domain classifier baseline on CLEAR-10 ($p = 0.034$), a false alarm of the kind that α allows.

5.5 Reliability and triage

The last two stages score individual predictions and advise on retraining. The fused risk score is judged by how well it ranks wrong predictions. Where the fusion is learned, on ACS and CLEAR-10, its area under the receiver operating characteristic curve (AUROC) differs from that of confidence alone by −0.012 to +0.001 over 72 windows and by −0.002 to +0.009 on the benchmark. On the gas sensor array the calibration part contains too few wrong predictions (4 with LR and 11 with GB), so the risk falls back to confidence. Confidence alone already carries most of the information about a single prediction.

A retrain truly helps when retraining with importance weights or on the latest labeled window significantly raises holdout accuracy. Over 82 real settings (Table S1 in the Supplement), the triage is right in 54 of the 58 settings that have an earlier labeled window, because its recent-window probe is validated on real labels. Without one it relies on importance-weighted validation and is right in 23 of 24 settings, but mostly by advising against retraining, since one truly helped in only 2 of them. On the benchmark its recall is 6%, because importance-weighted validation cannot see gains outside the reference data.

6 Discussion

The results support the central claim. A diagnosis built from tests of the type definitions transfers to drift it was not designed for, whereas a learned rule does not. The cost is a small label audit, needed only for concept drift and label noise.

Several limitations qualify these findings. Each benchmark window contains a single type, while real windows such as ACS Income 2023 mix several. The four types are also an idealization: sensor aging changes $P(X \mid Y)$ and, through it, $P(Y \mid X)$, yet our procedure reports it as covariate shift. Design B reduces, but cannot eliminate, the risk that the benchmark favors the tests. The weighted residual tests also ignore the sampling error of the BBSE weights. With hundreds of thousands of rows even small shifts become significant, which is why effect sizes are reported next to every decision. Deep models trained end to end, text and streaming data are not covered.

7 Conclusions

We showed that the drift type, which determines the response, can be identified by hypothesis tests with a stated error rate from label-free tests and a few hundred late labels. On injected drift the procedure outperforms rules from monitoring practice and the label shift literature, works for both model families and, unlike a learned meta-classifier, transfers to held-out drift. On real drift it reported no drift on any control window and traced documented events, from new occupation codes to aging sensors.

Drift that leaves accuracy intact passes the audit unnoticed, while class-conditional label noise is often read as concept drift. Future work will condition the audit on the full model output, model the sampling error of the BBSE weights and test mixed drift.

References

[1] Rabanser, S., Günnemann, S., Lipton, Z. C. "Failing Loudly: An Empirical Study of Methods for Detecting Dataset Shift", In: Advances in Neural Information Processing Systems 32, Vancouver, Canada, 2019, pp. 1394–1406. [online] Available at: https://proceedings.neurips.cc/paper_files/paper/2019/hash/846c260d715e5b854ffad5f70a516c88-Abstract.html [Accessed: 21 October 2026]

[2] Garg, S., Balakrishnan, S., Lipton, Z. C., Neyshabur, B., Sedghi, H. "Leveraging Unlabeled Data to Predict Out-of-Distribution Performance", In: The Tenth International Conference on Learning

Representations, Virtual, 2022. [online] Available at: https://arxiv.org/abs/2201.04234 [Accessed: 21 October 2026]

[3] Kivimäki, J., Białek, J., Kuberski, W., Nurminen, J. K. "Performance Estimation in Binary Classification Using Calibrated Confidence", Machine Learning, 115(3), 67, 2026. https://doi.org/10.1007/s10994-025-06970-3

[4] Moreno-Torres, J. G., Raeder, T., Alaiz-Rodríguez, R., Chawla, N. V., Herrera, F. "A Unifying View on Dataset Shift in Classification", Pattern Recognition, 45(1), pp. 521–530, 2012. https://doi.org/10.1016/j.patcog.2011.06.019

[5] Lipton, Z. C., Wang, Y.-X., Smola, A. J. "Detecting and Correcting for Label Shift with Black Box Predictors", In: Proceedings of the 35th International Conference on Machine Learning, Stockholm, Sweden, 2018, pp. 3122–3130. [online] Available at: https://proceedings.mlr.press/v80/lipton18a.html [Accessed: 21 October 2026]

[6] Roschewitz, M., Mehta, R., Jones, C., Glocker, B. "Automatic Dataset Shift Identification to Support Safe Deployment of Medical Imaging AI", In: Medical Image Computing and Computer Assisted Intervention – MICCAI 2025, Daejeon, South Korea, 2025, pp. 67–76. https://doi.org/10.1007/978-3-032-04981-0_7

[7] Maia Polo, F., Izbicki, R., Lacerda, E. G., Ibieta-Jimenez, J. P., Vicente, R. "A Unified Framework for Dataset Shift Diagnostics", Information Sciences, 649, 119612, 2023. https://doi.org/10.1016/j.ins.2023.119612

[8] Gama, J., Medas, P., Castillo, G., Rodrigues, P. "Learning with Drift Detection", In: Advances in Artificial Intelligence – SBIA 2004, São Luís, Brazil, 2004, pp. 286–295. https://doi.org/10.1007/978-3-540-28645-5_29

[9] Gretton, A., Borgwardt, K. M., Rasch, M. J., Schölkopf, B., Smola, A. "A Kernel Two-Sample Test", Journal of Machine Learning Research, 13, pp. 723–773, 2012. [online] Available at: https://jmlr.org/papers/v13/gretton12a.html [Accessed: 21 October 2026]

[10] Lopez-Paz, D., Oquab, M. "Revisiting Classifier Two-Sample Tests", In: 5th International Conference on Learning Representations, Toulon, France, 2017. [online] Available at: https://arxiv.org/abs/1610.06545 [Accessed: 21 October 2026]

[11] Hendrycks, D., Gimpel, K. "A Baseline for Detecting Misclassified and Out-of-Distribution Examples in Neural Networks", In: 5th International Conference on Learning Representations, Toulon, France, 2017. [online] Available at: https://arxiv.org/abs/1610.02136 [Accessed: 21 October 2026]

[12] Guillory, D., Shankar, V., Ebrahimi, S., Darrell, T., Schmidt, L. "Predicting with Confidence on Unseen Distributions", In: 2021 IEEE/CVF International Conference on Computer Vision, Montreal, QC, Canada, 2021, pp. 1114–1124. https://doi.org/10.1109/ICCV48922.2021.00117

[13] Ovadia, Y., Fertig, E., Ren, J., Nado, Z., Sculley, D., Nowozin, S., Dillon, J. V., Lakshminarayanan, B., Snoek, J. "Can You Trust Your Model's Uncertainty? Evaluating Predictive Uncertainty under Dataset Shift", In: Advances in Neural Information Processing Systems 32, Vancouver, Canada, 2019, pp. 13991–14002. [online] Available at: https://proceedings.neurips.cc/paper_files/paper/2019/hash/8558cb408c1d76621371888657d2eb1d-Abstract.html [Accessed: 21 October 2026]

[14] Lundberg, S. M., Lee, S.-I. "A Unified Approach to Interpreting Model Predictions", In: Advances in Neural Information Processing Systems 30, Long Beach, CA, USA, 2017, pp. 4765–4774. [online] Available at: https://proceedings.neurips.cc/paper_files/paper/2017/hash/8a20a8621978632d76c43dfd28b67767-Abstract.html [Accessed: 21 October 2026]

[15] Zhou, B., Khosla, A., Lapedriza, A., Oliva, A., Torralba, A. "Learning Deep Features for Discriminative Localization", In: 2016 IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 2016, pp. 2921–2929. https://doi.org/10.1109/CVPR.2016.319

[16] Liu, F. T., Ting, K. M., Zhou, Z.-H. "Isolation Forest", In: 2008 Eighth IEEE International Conference on Data Mining, Pisa, Italy, 2008, pp. 413–422. https://doi.org/10.1109/ICDM.2008.17

[17] Shimodaira, H. "Improving Predictive Inference under Covariate Shift by Weighting the Log-Likelihood Function", Journal of Statistical Planning and Inference, 90(2), pp. 227–244, 2000. https://doi.org/10.1016/S0378-3758(00)00115-4

[18] Sugiyama, M., Krauledat, M., Müller, K.-R. "Covariate Shift Adaptation by Importance Weighted Cross Validation", Journal of Machine Learning Research, 8, pp. 985–1005, 2007. [online] Available at: https://jmlr.org/papers/v8/sugiyama07a.html [Accessed: 21 October 2026]

[19] Cortes, C., Mansour, Y., Mohri, M. "Learning Bounds for Importance Weighting", In: Advances in Neural Information Processing Systems 23, Vancouver, Canada, 2010, pp. 442–450. [online] Available at: https://proceedings.neurips.cc/paper_files/paper/2010/hash/59c33016884a62116be975a9bb8257e3-Abstract.html [Accessed: 21 October 2026]

[20] Ding, F., Hardt, M., Miller, J., Schmidt, L. "Retiring Adult: New Datasets for Fair Machine Learning", In: Advances in Neural Information Processing Systems 34, Virtual, 2021, pp. 6478–6490. [online] Available at: https://proceedings.neurips.cc/paper_files/paper/2021/hash/32e54441e6382a7fbacbbbaf3c450059-Abstract.html [Accessed: 21 October 2026]

[21] Lin, Z., Shi, J., Pathak, D., Ramanan, D. "The CLEAR Benchmark: Continual LEArning on Real-World Imagery", In: Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks 1, Virtual, 2021. [online] Available at: https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/2838023a778dfaecdc212708f721b788-Abstract-round2.html [Accessed: 21 October 2026]

[22] Holm, S. "A Simple Sequentially Rejective Multiple Test Procedure", Scandinavian Journal of Statistics, 6(2), pp. 65–70, 1979. [online] Available at: https://www.jstor.org/stable/4615733 [Accessed: 21 October 2026]

[23] He, K., Zhang, X., Ren, S., Sun, J. "Deep Residual Learning for Image Recognition", In: 2016 IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 2016, pp. 770–778. https://doi.org/10.1109/CVPR.2016.90

[24] Siddiqi, N. "Credit Risk Scorecards: Developing and Implementing Intelligent Credit Scoring", Wiley, Hoboken, NJ, USA, 2006. ISBN 978-0-471-75451-0 https://doi.org/10.1002/9781119201731

[25] Cochran, W. G. "Some Methods for Strengthening the Common χ^2 Tests", Biometrics, 10(4), pp. 417–451, 1954. https://doi.org/10.2307/3001616

[26] Phipson, B., Smyth, G. K. "Permutation P-values Should Never Be Zero: Calculating Exact P-values When Permutations Are Randomly Drawn", Statistical Applications in Genetics and Molecular Biology, 9(1), 39, 2010. https://doi.org/10.2202/1544-6115.1585

[27] Kish, L. "Survey Sampling", Wiley, New York, NY, USA, 1965.

[28] Peduzzi, P., Concato, J., Kemper, E., Holford, T. R., Feinstein, A. R. "A Simulation Study of the Number of Events per Variable in Logistic Regression Analysis", Journal of Clinical Epidemiology, 49(12), pp. 1373–1379, 1996. https://doi.org/10.1016/S0895-4356(96)00236-3

[29] Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M., Duchesnay, É. "Scikit-learn: Machine Learning in Python", Journal of Machine Learning Research, 12, pp. 2825–2830, 2011. [online] Available at: https://jmlr.org/papers/v12/pedregosa11a.html [Accessed: 21 October 2026]

[30] Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q., Liu, T.-Y. "LightGBM: A Highly Efficient Gradient Boosting Decision Tree", In: Advances in Neural Information Processing Systems 30, Long Beach, CA, USA, 2017, pp. 3146–3154. [online] Available at: https://proceedings.neurips.cc/paper_files/paper/2017/hash/6449f44a102fde848669bdd9eb6b76fa-Abstract.html [Accessed: 21 October 2026]

[31] U.S. Census Bureau "2020 ACS 1-Year Experimental Data Release", [online] Available at: https://www.census.gov/programs-surveys/acs/data/experimental-data.html [Accessed: 21 October 2026]

[32] U.S. Census Bureau "American Community Survey 2019 PUMS Data Dictionary", [online] Available at: https://www2.census.gov/programs-surveys/acs/tech_docs/pums/data_dict/PUMS_Data_Dictionary_2019.csv [Accessed: 21 October 2026]

[33] Beckhusen, J. B. "Recent Changes in the Census Industry and Occupation Classification Systems", U.S. Census Bureau, Suitland, MD, USA, Rep. ACS-TP78, 2020. [online] Available at: https://www.census.gov/library/publications/2020/demo/acs-tp78.html [Accessed: 21 October 2026]

[34] U.S. Census Bureau "Improvements to the Race Question (ACS User Note 2021-03)", [online] Available at: https://www.census.gov/programs-surveys/acs/technical-documentation/user-notes/2021-03.html [Accessed: 21 October 2026]

[35] Vergara, A., Vembu, S., Ayhan, T., Ryan, M. A., Homer, M. L., Huerta, R. "Chemical Gas Sensor Drift Compensation Using Classifier Ensembles", Sensors and Actuators B: Chemical, 166–167, pp. 320–329, 2012. https://doi.org/10.1016/j.snb.2012.01.074

[36] Rodriguez-Lujan, I., Fonollosa, J., Vergara, A., Homer, M., Huerta, R. "On the Calibration of Sensor Arrays for Pattern Recognition Using the Minimal Number of Experiments", Chemometrics and Intelligent Laboratory Systems, 130, pp. 123–134, 2014. https://doi.org/10.1016/j.chemolab.2013.10.012

[37] Wilson, E. B. "Probable Inference, the Law of Succession, and Statistical Inference", Journal of the American Statistical Association, 22(158), pp. 209–212, 1927. https://doi.org/10.1080/01621459.1927.10502953